

Demonstration: one population, five ways to sample it

Rohan Alexander

9 October, 2026

Introduction

With real data we get one sample, and we never see the population. That makes it hard to build intuition for what a sampling design does. So we do what we always do, which is simulate. We build a city where we know everything about everyone, sample it many times under different designs, and see how the estimates behave.

The question is the one Bowley (1913) asked about Reading, which is what proportion of households are in poverty. The designs are those in Chapter 6 of Alexander (2023), which are simple random, systematic, stratified, and cluster sampling, and then a convenience sample for comparison.

As in Weeks 1 and 2, Python (Astral 2025; Vink et al. 2025) does the simulation and saves Parquet files, and R (R Core Team 2025; Richardson et al. 2025; Wickham 2016; Arel-Bundock 2025) reads them to make graphs and tables.

Step 1: simulate a population

The city has 20 wards. Each ward has 25 blocks, and each block has 200 households, so there are 100,000 households. Households in the same block tend to have similar incomes, blocks in the same ward tend to be similar, and wards differ. That structure matters because it is what makes stratifying help and clustering hurt. A household is in poverty if its weekly income is below \$600.

```
import numpy as np
import polars as pl

rng = np.random.default_rng(seed=853)
```

```

NUM_WARDS = 20
BLOCKS_PER_WARD = 25
HOUSEHOLDS_PER_BLOCK = 200
POVERTY_LINE = 600 # dollars a week

# Wards differ from each other, blocks differ within a ward, and households
# differ within a block. All on the log scale, so income is right-skewed.
ward_effect = rng.normal(loc=0, scale=0.35, size=NUM_WARDS)
block_effect = rng.normal(loc=0, scale=0.25, size=(NUM_WARDS, BLOCKS_PER_WARD))

```

The rest of `scripts/00-simulate_population.py` builds the data frame, tests it (the number of rows, wards, and blocks are what we planned, income is positive, and the household identifier is unique), and writes `data/population.parquet`. Run it with:

```
uv run scripts/00-simulate_population.py
```

```

library(arrow)
library(dplyr)
library(ggplot2)
library(tinytable)

population <- read_parquet("data/population.parquet")

glimpse(population)

```

```

Rows: 100,000
Columns: 5
$ household    <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
$ ward         <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ block        <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ weekly_income <dbl> 2454, 2104, 1926, 1038, 753, 2113, 2402, 3135, 1511, 291~
$ in_poverty   <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, ~

```

```

truth <- mean(population$in_poverty)
truth

```

```
[1] 0.24064
```

So the number that every design is trying to estimate is 24.1 per cent. In practice this is the number we never get to see.

Figure 1 shows why the design is going to matter. The poverty rate differs a great deal between wards, and between blocks within a ward.

```
population |>
  summarise(poverty_rate = mean(in_poverty), .by = c(ward, block)) |>
  ggplot(aes(x = factor(ward), y = poverty_rate)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = truth) +
  labs(x = "Ward", y = "Proportion of households in poverty") +
  theme_minimal()
```

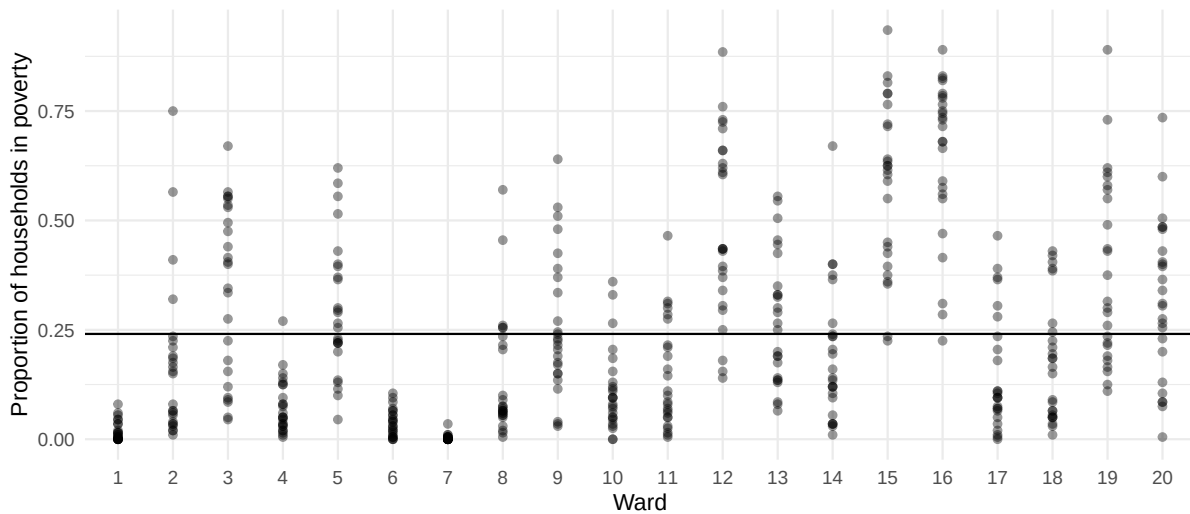


Figure 1: Poverty rate of each of the 500 blocks, by ward. Each point is a block of 200 households. The line is the rate for the whole city.

Step 2: sample it, many times

Each design gets the same budget of 1,000 households.

- Simple random: 1,000 households from the whole list.
- Systematic: the list is ordered by ward and block, as a street directory would be. Pick a random start in the first 100, then take every 100th household.
- Stratified by ward: 50 households at random from each of the 20 wards.
- Two-stage cluster: 25 blocks at random, then 40 households at random in each.
- Convenience: 1,000 households from the two wards nearest the office.

We draw 1,000 samples under each design and keep the estimate from each. The core of `scripts/01-sample.py` is one small function per design, for instance:

```
def stratified():
    # Proportional allocation: every ward is the same size, so 50 from each.
    # With equal-sized strata the weights are equal and the mean of means is fine.
    per_ward = n // len(wards)
    means = []
    for w in wards:
        chosen = rng.choice(index_by_ward[w], size=per_ward, replace=False)
        means.append(in_poverty[chosen].mean())
    return np.mean(means)

def two_stage_cluster():
    # 25 blocks at random, then 40 households within each
    chosen_blocks = rng.choice(blocks, size=25, replace=False)
    means = []
    for b in chosen_blocks:
        chosen = rng.choice(index_by_block[b], size=40, replace=False)
        means.append(in_poverty[chosen].mean())
    return np.mean(means)
```

```
uv run scripts/01-sample.py
```

Step 3: compare

```
estimates <- read_parquet("data/estimates.parquet") |>
  mutate(design = factor(design, levels = unique(design)))
```

```
estimates |>
  ggplot(aes(x = estimate)) +
  geom_histogram(binwidth = 0.005) +
  geom_vline(xintercept = truth) +
  facet_wrap(vars(design), ncol = 1, scales = "free_y") +
  labs(
    x = "Estimated proportion of households in poverty",
    y = "Number of samples"
  ) +
  theme_minimal()
```

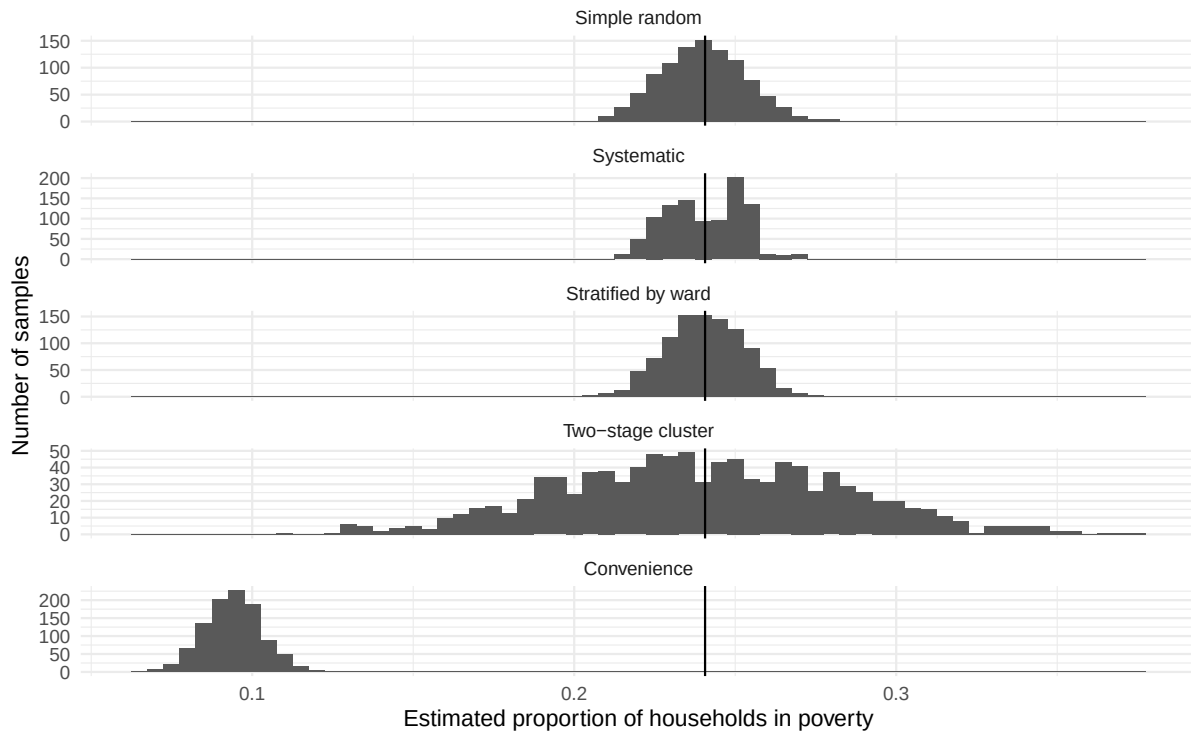


Figure 2: Estimates of the poverty rate from 1,000 samples of 1,000 households under each design. The vertical line is the truth.

```
estimates |>
  summarise(
    Mean = mean(estimate),
    SD = sd(estimate),
    RMSE = sqrt(mean((estimate - truth)^2)),
    .by = design
  ) |>
  rename(Design = design) |>
  tt() |>
  format_tt(j = 2:4, digits = 3, num_fmt = "decimal", num_zero = TRUE)
```

What to take from Figure 2 and Table 1:

1. Simple random sampling is centered on the truth, with a standard deviation of about 0.013. That is what the formula says it should be, because $\sqrt{0.24 \times 0.76/1000} \approx 0.0135$. The standard error that we compute from one sample is an estimate of the spread of this histogram, which we ordinarily never see.
2. Stratifying by ward does a little better for the same budget, because wards differ and we have removed the chance of getting a sample with too many households from a poor ward.

Table 1: Mean, standard deviation, and root mean squared error of the estimates under each design

Design	Mean	SD	RMSE
Simple random	0.240	0.013	0.013
Systematic	0.241	0.012	0.012
Stratified by ward	0.241	0.012	0.012
Two-stage cluster	0.240	0.045	0.045
Convenience	0.094	0.009	0.147

Systematic sampling from a list that is ordered by ward gets that same benefit without us asking for it, because it is implicitly stratified. (There are only 100 possible systematic samples here, one for each starting point, which is why that histogram is lumpier.)

3. The cluster sample is centered on the truth but is more than three times as variable. Households in a block are alike, so 40 households from one block carry much less information than 40 households from 40 blocks. This sample of 1,000 is worth about as much as a simple random sample of 90. We would only do this if it were much cheaper, and with interviewers walking door-to-door it is.
4. The convenience sample is the most precise of all, and it is wrong. It says nine per cent when the truth is 24 per cent. A larger convenience sample would give the same wrong answer with even more confidence. Nothing in the sample itself tells you this. Only knowing how it was collected does.

The last point is the one to hold on to for the rest of the course. Most of the datasets that we actually work with are closer to the convenience sample than to any of the others.

Things to change

- Make the wards identical (set the scale of `ward_effect` to zero). What happens to the advantage of stratifying?
- Make the blocks within a ward identical. What happens to the penalty for clustering?
- Hold the budget at 1,000 but take 100 blocks of 10 households, and then 10 blocks of 100. Which is better, and what would each cost?
- Sort the list by income before taking the systematic sample. Then sort it so that every 100th household is the richest in its block.
- Stratify, but sample 100 from each of the five most variable wards and fewer from the rest, as Neyman (1934) suggests. Remember to weight.

References

- Alexander, Rohan. 2023. *Telling Stories with Data*. Chapman and Hall/CRC. <https://tellingswithdata.com>.
- Arel-Bundock, Vincent. 2025. *tinytable: Simple and Configurable Tables in HTML, LaTeX, Markdown, Word, PNG, PDF, and Typst Formats*. <https://vincentarelbundock.github.io/tinytable/>.
- Astral. 2025. *uv: An Extremely Fast Python Package and Project Manager*. <https://docs.astral.sh/uv/>.
- Bowley, Arthur Lyon. 1913. “Working-Class Households in Reading.” *Journal of the Royal Statistical Society* 76 (7): 672–701. <https://doi.org/10.2307/2339708>.
- Neyman, Jerzy. 1934. “On the Two Different Aspects of the Representative Method: The Method of Stratified Sampling and the Method of Purposive Selection.” *Journal of the Royal Statistical Society* 97 (4): 558–625. <https://doi.org/10.2307/2342192>.
- R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>.
- Richardson, Neal, Ian Cook, Nic Crane, et al. 2025. *arrow: Integration to Apache Arrow*. <https://arrow.apache.org/docs/r/>.
- Vink, Ritchie et al. 2025. *Polars: DataFrames for the New Era*. <https://pola.rs>.
- Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>.